SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1504

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 50**
Question Count: 5

Code of Conduct

I **P Dinesh Reddy(192525399** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.

   Design an HDFS-based storage solution for a media platform

   Proposed Design

   The media platform can use **HDFS with a replication factor of 3** to store videos, images, and other large media files. The architecture consists of a **NameNode**, multiple **DataNodes**, and client applications.

   Architecture:

   Media Application → NameNode → DataNodes

   - **NameNode:** Maintains file names, metadata, and block locations.
   - **DataNodes:** Store the actual media data blocks.
   - **Replication:** Each data block is replicated on **3 DataNodes**.
   - **Client:** Uploads and retrieves media files through HDFS.

   Constraint Handling

   1. **Data availability above 99.9%**
      - Use **3-way replication** to provide redundancy.
      - Store replicas across different DataNodes/racks.
      - If one DataNode fails, another replica can serve the data.
      - NameNode high availability can also be used to avoid a single point of failure.
   2. **Replication factor ≤ 3**
      - Set the HDFS replication factor to exactly **3**.
      - This provides good fault tolerance without exceeding the specified constraint.

3. **Cost-efficient storage growth**
    - Add DataNodes whenever storage demand increases.
    - Use commodity servers instead of expensive specialized storage.
    - Store large media files in HDFS blocks to utilize storage efficiently.
    - Monitor storage utilization and add capacity only when required.

Justification

This design satisfies all three constraints. **Three-way replication** provides high availability and fault tolerance, while the replication factor remains within the limit. HDFS also supports **horizontal scaling**, allowing storage capacity to grow by adding DataNodes. Therefore, the solution provides a reliable, scalable, and cost-efficient storage platform for media data.

2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

MapReduce Workflow for Log Data Processing

Proposed Design

A **MapReduce** job can process the log data in parallel across the limited cluster nodes. The workflow consists of **Input → Mapper → Shuffle & Sort → Reducer → Output**.

Architecture:

Log Files → Input Splits → Mappers → Shuffle & Sort → Reducers → Output

Job Design

- **Input:** Divide large log files into multiple input splits.
- **Mapper:** Each mapper reads log records and extracts required information such as timestamps, IP addresses, error codes, or request counts.
- **Shuffle & Sort:** Groups intermediate key-value pairs so that related records are sent to the same reducer.
- **Reducer:** Aggregates the grouped data and generates the final results.
- **Output:** Store the processed results in HDFS.

Task Distribution

1. Divide the log dataset into **multiple input splits**.
2. Assign each split to an available **Mapper task**.
3. Run mapper tasks **in parallel** on the limited cluster nodes.
4. Distribute intermediate data among reducers using **Shuffle and Sort**.
5. Execute reducers in parallel and write the final output to HDFS.

Constraint Handling

- **20-minute batch window:** Use parallel mapper and reducer execution to reduce processing time.
- **Limited nodes:** Assign multiple tasks to each node while balancing CPU, memory, and I/O resources.
- **Efficient processing:** Prefer data locality so that computation occurs close to the stored HDFS data.
- **Load balancing:** Distribute input splits evenly to prevent individual nodes from becoming bottlenecks.

Justification

The proposed MapReduce workflow uses **parallel task execution, data locality, and balanced task distribution** to process log data efficiently within the **20-minute batch constraint**, even with limited cluster nodes.

3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

Design a YARN Scheduler Policy

Proposed Design

A **YARN-managed cluster** can use a **Capacity Scheduler** with separate queues for ETL, reporting, and machine learning jobs.

Architecture:

Applications → YARN ResourceManager → Scheduler → NodeManagers

Queue Design

- **ETL Queue:** Highest priority because ETL jobs are time-sensitive.
- **Reporting Queue:** Medium priority for regular business reports.
- **Machine Learning Queue:** Lower priority, allowing long-running ML jobs to use available resources.

Constraint Handling

1. **Fairness**
    - Allocate minimum resources to each queue.
    - Prevent one workload from consuming all cluster resources.
    - Share unused resources among queues when available.
2. **Priority for ETL**
    - Give ETL jobs higher scheduling priority.
    - Allocate available resources to ETL first when necessary.
    - Prevent lower-priority workloads from delaying critical ETL processing.
3. **Recovery from Node Failure**
    - YARN detects failed NodeManagers through heartbeats.
    - Failed tasks are rescheduled on healthy nodes.
    - Replicated HDFS data allows tasks to access available copies of required data.

Justification

The proposed YARN policy provides **fair resource sharing**, gives **higher priority to ETL workloads**, and supports **automatic task recovery after node failures**. This ensures efficient and reliable execution of ETL, reporting, and machine learning workloads.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

Data Ingestion from SAN, NAS, and Operational Databases

Proposed Architecture

The enterprise can use a **big data ingestion layer** to collect data from SAN, NAS, and operational databases and store it in a centralized Hadoop platform.

Architecture:

SAN ─┐
NAS ─┼→ Ingestion Layer → HDFS → Processing → Analytics
DB ─┘

Components

- **SAN:** Provides high-performance access to enterprise storage data.
- **NAS:** Supplies file-based data such as documents and media.
- **Operational Databases:** Provide structured transactional data.
- **Ingestion Layer:** Collects data from all sources and transfers it to the Hadoop platform.
- **HDFS:** Provides centralized and replicated storage.
- **Backup Mechanism:** Maintains reliable copies of critical data.

Constraint Handling

1. **Minimal Duplication**
    - Use a centralized ingestion process to avoid unnecessary copies.
    - Maintain a common data format and metadata for incoming datasets.
    - Store each dataset in its appropriate HDFS location.
2. **Backup Reliability**
    - Use HDFS replication for fault tolerance.
    - Maintain backup copies of important datasets.

- Monitor data integrity during ingestion and storage.

3. **Reliable Data Ingestion**
   - Use appropriate connectors for SAN, NAS, and databases.
   - Schedule ingestion jobs according to data source requirements.
   - Validate data before storing it in HDFS.

Justification

This architecture integrates **SAN, NAS, and operational databases** into a centralized Hadoop environment while controlling unnecessary duplication and maintaining reliable backups. HDFS provides scalable and fault-tolerant storage for the ingested data.

5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Design an End-to-End Hadoop Ecosystem Solution

Proposed Architecture

The solution integrates **Apache NiFi/ETL pipelines, HDFS, YARN, and Hadoop processing** to securely move, store, and process enterprise data.

Architecture:

Data Sources → Apache NiFi / ETL → Secure Transfer → HDFS → YARN → MapReduce / Analytics

Components

- **Apache NiFi:** Collects and routes data from different sources.
- **Secure Data Transfer:** Uses authentication and encryption to protect data during movement.
- **HDFS:** Provides distributed, replicated storage for the collected data.
- **YARN:** Manages cluster resources and schedules processing applications.
- **MapReduce/ETL:** Processes and transforms the stored data.
- **Monitoring:** Tracks data flows, node health, and processing failures.

Constraint Handling

1. **Secure Data Movement**
   - Authenticate data sources and users.
   - Encrypt data during transfer.
   - Apply access controls to sensitive datasets.
2. **High Availability**
   - Use HDFS replication to maintain multiple copies of data.
   - Use redundant Hadoop services to reduce single points of failure.
   - Reschedule failed processing tasks on healthy nodes.

3. **NiFi/ETL Integration**
    - NiFi collects, transforms, and routes incoming data.
    - ETL pipelines clean and prepare data before storage or analysis.
    - Processed data is stored in HDFS for further Hadoop processing.

Justification

The proposed architecture provides **secure data movement, highly available storage, and seamless NiFi/ETL integration**. HDFS ensures reliable distributed storage, while YARN manages processing resources and supports recovery from failures. Thus, the design satisfies the major constraints of the Hadoop ecosystem solution.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured